

Week 1 - NUnit & Moq Hands-on Answers

1. NUnit Hands-on - Calculator Addition Test

CalculatorTests.cs

```
using NUnit.Framework;
using CalcLibrary;

[TestFixture]
public class CalculatorTests
{
    private Calculator calc;

    [SetUp]
    public void Init()
    {
        calc = new Calculator();
    }

    [TearDown]
    public void Cleanup()
    {
        calc = null;
    }

    [Test]
    [TestCase(10, 20, 30)]
    [TestCase(5, 7, 12)]
    [TestCase(-5, 5, 0)]
    public void TestAddition(int a, int b, int expected)
    {
        int actual = calc.Add(a, b);
        Assert.That(actual, Is.EqualTo(expected));
    }
}
```

2. Moq Hands-on - Write Testable Code with Moq

CustomerCommTests.cs

```
using NUnit.Framework;
using Moq;
using CustomerCommLib;

[TestFixture]
public class CustomerCommTests
{
    private Mock<IMailSender> mockMail;
    private CustomerComm customerComm;

    [OneTimeSetUp]
    public void Setup()
```

```
{
    mockMail = new Mock<IMailSender>();

    mockMail.Setup(x => x.SendMail(
        It.IsAny<string>(),
        It.IsAny<string>()))
        .Returns(true);

    customerComm = new CustomerComm(mockMail.Object);
}

[TestCase]
public void SendMailToCustomer_ReturnsTrue()
{
    bool result = customerComm.SendMailToCustomer();

    Assert.That(result, Is.True);
}
}
```